

Software Requirements Specification (SRS)

DevSync - Team Coordination & Notification Platform

Document Version: 1.0

Date: March 27, 2026

Project Name: DevSync

Project Type: Web & Mobile Application

Target Users: Software Development Teams

Table of Contents

- [1. Introduction](#)
 - [2. Overall Description](#)
 - [3. Specific Requirements](#)
 - [4. UI/UX Specifications](#)
 - [5. Workflows & Use Cases](#)
 - [6. Data Models](#)
 - [7. Non-Functional Requirements](#)
 - [8. Assumptions & Constraints](#)
-

1. Introduction

1.1 Purpose

DevSync is a centralized platform designed to improve team coordination during software development. It eliminates scattered communication across WhatsApp and integrates with GitHub to provide real-time notifications, team communication, and lightweight task management in one place.

1.2 Problem Statement

Development teams currently use multiple tools:

- GitHub:** For code management and PRs
- WhatsApp:** For notifications and informal discussion

Issues:

- Important review requests get lost in chat history
- No centralized record of decisions
- Context switching between apps wastes time
- Difficult to track who's responsible for what
- Notifications are unpredictable

1.3 Solution Overview

DevSync acts as a **notification and coordination hub** between GitHub and the team:

- Automatic email alerts when code is pushed
- In-app discussion threads for each PR
- Light task management system
- Complete activity history
- Professional, organized workflow

1.4 Scope

In Scope:

- GitHub webhook integration (PR and push notifications)
- Email notification system
- Web application for desktop access
- Mobile app for iOS/Android access
- In-app messaging/comments
- Task assignment system
- User authentication & authorization
- Activity feed and history

Out of Scope:

- Code review comments on GitHub (use GitHub for this)
- Complex issue tracking (GitHub Issues handles this)
- PR merging from DevSync (merge on GitHub)
- Code diff viewing (view on GitHub)
- CI/CD pipeline management

2. Overall Description

2.1 System Overview

```
GitHub Repository
  ↓ (Webhook Event)
DevSync Backend Server
  ↓ (Process & Store)
Database
  ↓ (Notify)
Email Service → User Email
  ↓
Web/Mobile App → User Access
  ↓
User Chat & Discussion
```

2.2 Product Perspective

DevSync is a standalone SaaS platform that:

- Integrates with external GitHub repositories
- Sends emails via SendGrid/AWS SES
- Stores data in cloud database
- Accessible via web browser and mobile app
- Uses GitHub OAuth for authentication

2.3 User Classes

2.3.1 Team Members (Regular Users)

- Access pull requests
- Receive email notifications
- Chat about PRs
- View assigned tasks
- Update task status
- View activity feed

2.3.2 Team Admins

- All Member permissions
- Add/remove team members
- Configure GitHub repo connection
- Set notification preferences for team
- View team analytics

2.3.3 Project Owner

- Create teams/projects
- Manage billing (future)
- Delete projects

- Super admin access
-

3. Specific Requirements

3.1 Functional Requirements

FR1: User Authentication & Authorization

FR1.1 User Registration

- Email and password based signup
- Verification email sent after signup
- User must verify email before accessing platform
- Password must be strong (min 8 chars, numbers, symbols)

FR1.2 User Login

- Email and password login
- Option: GitHub OAuth login (recommended)
- Remember me functionality
- Forgot password link
- Session timeout after 30 minutes of inactivity

FR1.3 User Profile

- View/edit user profile (name, email, avatar)
- Change password
- Set notification preferences
- View connected GitHub account
- Disconnect GitHub account

FR1.4 Authorization

- Only team members can access team data
- Admins can manage team settings
- Users can only edit their own profile
- Access control at API level (backend enforces)

FR2: GitHub Integration

FR2.1 Repository Connection

- Admin can link GitHub repo to DevSync project
- One-time setup (4 steps):
 1. Copy webhook secret from DevSync
 2. Go to GitHub repo → Settings → Webhooks

3. Paste webhook URL: `https://devsync.com/api/webhook/github`

4. Select events: "push" and "pull_request" events

- Confirmation message when successful
- Can disconnect repo anytime
- Support multiple repos per team

FR2.2 Receive GitHub Events

- Listen for webhooks from GitHub
- Events supported:
 - `pull_request.opened` → New PR created
 - `pull_request.synchronize` → New commit pushed to PR
 - `pull_request.closed` → PR closed
 - `pull_request.merged` → PR merged
 - `push` → Code pushed to any branch
- Validate webhook signature (security)
- Retry mechanism if delivery fails

FR2.3 Parse & Store Event Data Extract and store:

- Pull Request ID (GitHub ID)
 - Branch name
 - PR title and description
 - Author name and avatar
 - Commits count
 - Changed files count
 - Creation timestamp
 - PR URL on GitHub
-

FR3: Email Notification System

FR3.1 Event-Based Notifications Trigger email when:

- New PR is created → All team members
- PR needs review → Assigned reviewers
- PR is merged → All team members
- New comment on user's PR → Author
- User is mentioned in comment → Mentioned user
- Task assigned to user → Assigned user

FR3.2 Email Template & Content**

Email Subject: "[DevSync] New PR: Feature Name by Author"

Email Body:

Hi {{username}},

{{Author}} has created a new pull request:

□ PR Title: {{title}}

□ Branch: {{branch_name}}

□ Author: {{author_name}}

□ Created: {{timestamp}}

Quick Links:

□ View on GitHub: {{github_link}}

□ Discuss in DevSync: {{devsync_link}}

What's happening:

{{description_first_100_chars}}...

Next Steps:

1. Review the code on GitHub
2. Add your comments in DevSync
3. Merge when ready

Manage notifications: {{preferences_link}}

FR3.3 Notification Preferences** Users can choose:

- Receive all notifications (default)
- Receive only assigned PRs
- Daily digest instead of instant
- Turn off specific notification types
- Email address to receive notifications

FR3.4 Smart Digest Option (v2)**

- Instead of sending instant emails for every event
- Collect events and send daily digest
- Example: "5 PRs created today, 2 merged, 3 awaiting review"
- Send digest at specific time (e.g., 9 AM)

FR4: Pull Request Management

FR4.1 View Pull Requests**

- Display list of all PRs from connected repos
- Show: PR number, title, author, branch, created date
- Filter by: status (open/merged), author, repo
- Sort by: newest first, oldest first, most active
- Search by PR title or branch name
- Pagination (20 PRs per page)

FR4.2 PR Details**

- Click PR → Open full details
- Show GitHub link (opens in new tab)
- Show all metadata: title, description, author, branch, date
- Display commit count and changed files count
- Show PR status: Open / Merged / Closed
- Click "View on GitHub" → Opens GitHub PR in new tab

FR4.3 PR Comments & Discussion**

- Click PR → See discussion thread
- Display all comments sorted by date
- Show who commented, when, and what they said
- Add new comment (text input)
- Comment appears instantly in app
- Comments stored in DevSync database
- **Not** the same as GitHub comments (use GitHub for code review)
- Mention users with @username
- Rich text: bold, italic, code blocks

FR4.4 PR Status Tracking**

- DevSync updates PR status when GitHub updates
- Status: Open → Merged / Closed
- Show "merged at" timestamp
- Show who merged it

FR5: In-App Messaging & Discussion

FR5.1 PR Discussion Thread**

- Every PR has a dedicated discussion page
- Example: "Feature/login-fix - Discussion"
- Timeline view of all messages
- Each message shows: author, timestamp, avatar, message text

FR5.2 Sending Messages**

- Text input box with "Send" button
- Support @ mentions (@dev, @all, etc.)
- Support markdown formatting:
 - **Bold** text
 - *Italic* text
 - Code snippets
 - Links
- Character limit: 5000 characters
- Emojis supported ☺
- Send via Enter or Send button

FR5.3 Message Features**

- Edit own message within 5 minutes
- Delete own message
- See "edited" indicator if message is modified
- See message timestamp (e.g., "2 hours ago")
- Real-time updates (if other user sends message, it appears instantly)

FR5.4 Notifications on Messages**

- Notify author when someone comments on their PR
- Notify mentioned users when @mentioned
- Notification email sent instantly (or per preference)
- In-app badge showing unread messages

FR6: Simple Task Management

FR6.1 Create Tasks**

- Admin/creator can create task in team
- Task fields:
 - Title (required)
 - Description (optional)
 - Assigned to (required)
 - Priority: Low / Medium / High
 - Due date (optional)
 - Status: To Do / In Progress / Done
 - Related PR (optional)
- Maximum 50 tasks per project

FR6.2 Task Assignment**

- Click "Create Task" button
- Select team member to assign to
- Assignee gets email notification
- Assigned user can see task in their dashboard

FR6.3 Task Status Updates**

- Assigned user can change task status
- Drag-and-drop between columns (Kanban board)
- Or click task → Change status dropdown
- Status change triggers notification
- History shows who changed status and when

FR6.4 Task Visibility**

- Show assigned tasks on personal dashboard
- Show all team tasks on project view
- Filter tasks by: assigned to, status, priority
- Search tasks by title
- Show task count on main dashboard

FR6.5 Task Notifications**

- Email when task assigned to you
 - Email when task status changes
 - Email reminder 1 day before due date
 - Daily digest of overdue tasks
-

FR7: Activity Feed & History

FR7.1 Activity Feed**

- Central timeline of all team activities
- Show events:
 - PR created: "Author created PR: Title"
 - PR merged: "Author merged PR #23"
 - Comment added: "Author commented on PR #23"
 - Task created: "Admin created task: XYZ"
 - Task status changed: "Member moved task XYZ to Done"
 - Branch pushed: "Author pushed to feature/xyz"
- Each event shows: timestamp, actor, action, details
- Sorted by newest first

FR7.2 Activity Filtering**

- Filter by: type (PR/task/comment), actor, date range
- Show: last 7 days, last 30 days, all time
- Search: find specific events

FR7.3 Notifications Log**

- Users can see notification history
- Show: what was notified, when, via which channel (email)

- Mark as read/unread in app
 - Archive old notifications
-

FR8: Team Management

FR8.1 Team Creation**

- First user (creator) becomes project owner
- Set team name, description
- Add GitHub repository (webhook setup)
- Team created successfully

FR8.2 Add Team Members**

- Admin can invite members via:
 - Email address
 - GitHub username
 - Shareable invite link
- Invited user gets email with signup link
- User signs up → Auto-added to team
- Max 50 team members per team

FR8.3 Remove Team Members**

- Admin can remove member from team
- Removed user loses access to team
- Their tasks are reassigned or marked as unassigned
- Their PR comments remain visible

FR8.4 Team Settings**

- Team admin can configure:
 - Team name, description, avatar
 - GitHub repository details
 - Notification settings (global)
 - Member list and roles
 - Delete team (owner only, requires confirmation)
-

3.2 Non-Functional Requirements

NFR1: Performance

- Page load time: < 2 seconds
- Email delivery: < 5 minutes
- Webhook processing: < 1 second
- Database queries: < 500ms
- Support 10,000+ PRs per team

- Support 100+ concurrent users

NFR2: Reliability

- 99.9% uptime SLA
- Automatic backup every 24 hours
- Disaster recovery plan
- Webhook retry logic (exponential backoff)
- Graceful error handling

NFR3: Security

- All data encrypted in transit (HTTPS/TLS)
- Passwords hashed with bcrypt
- GitHub webhook signature validation
- Rate limiting on APIs (100 requests/minute per user)
- No sensitive data in logs
- GDPR compliant (data deletion on request)
- SQL injection prevention
- XSS protection

NFR4: Scalability

- Horizontal scaling (add servers as needed)
- Database replication for high availability
- CDN for static assets
- Caching layer (Redis) for frequently accessed data
- Async job queue for email sending

NFR5: Usability

- Intuitive UI, no training required
- Mobile responsive
- Accessible (WCAG 2.1 AA)
- Support dark mode (v2)
- Multiple language support (v2)

NFR6: Maintainability

- Clean, documented code
- Unit tests (80% coverage)
- Integration tests
- API documentation
- Deployment automation

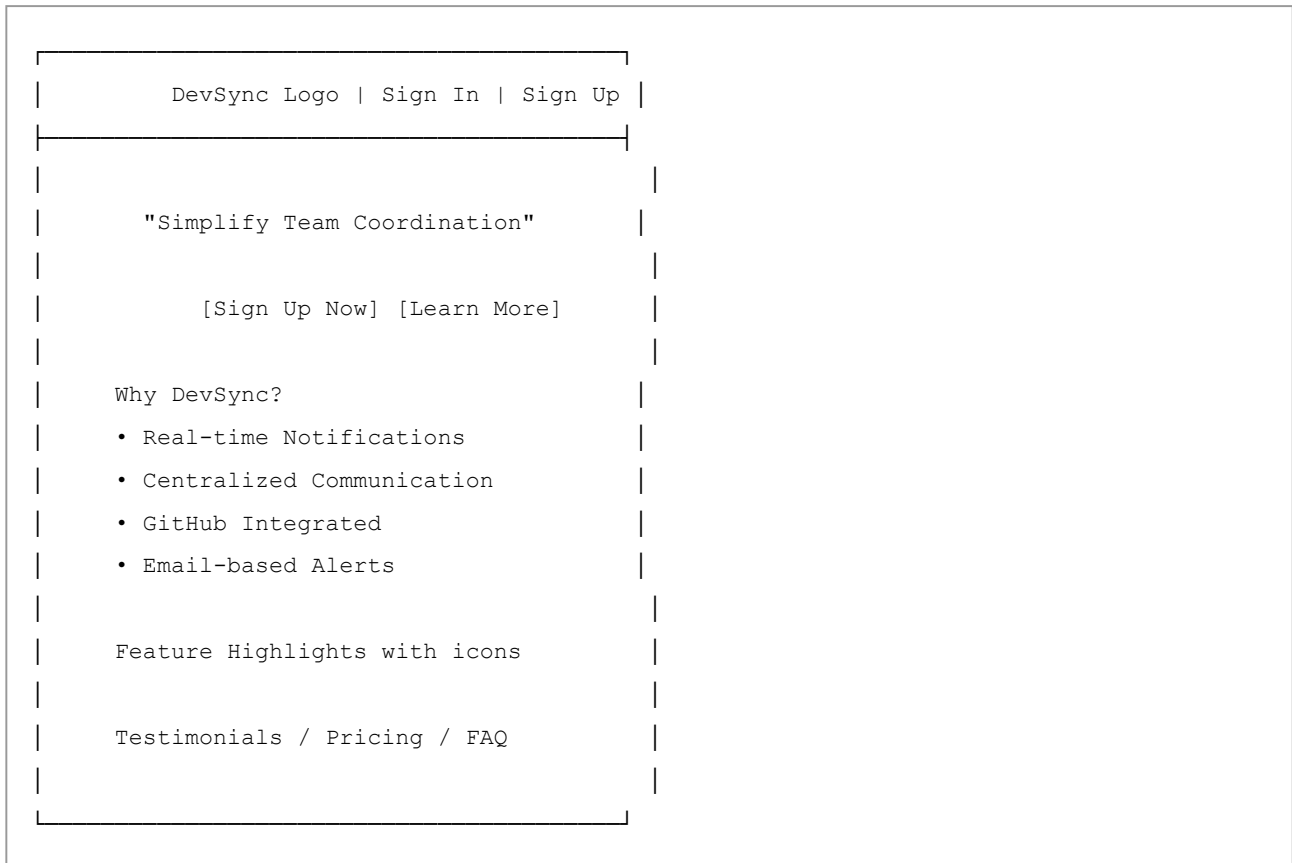
4. UI/UX Specifications

4.1 Web Application Pages

4.1.1 Landing Page (Before Login)

URL: /

Layout:



Components:

- Hero section with CTA buttons
- Features section (4-6 key features)
- How it works section
- FAQ section
- Footer with links

Elements:

- Navigation bar (sticky)
- Email signup for updates
- Social media links
- Privacy policy / Terms link

4.1.2 Sign Up / Registration Page

URL: /signup

Layout:

DevSync	Create Account
Logo	
	Full Name:
	[_____]
	Email:
	[_____]
	Password:
	[_____]
	(min 8 chars)
	[Sign Up]
	or via [GitHub OAuth]
	Already have account?
	[Sign In]

Form Fields:

- Full Name (text input)
- Email (email input, validation)
- Password (password input, strength indicator)
- Confirm Password
- Terms & Privacy checkbox
- Sign Up button
- OAuth button (GitHub)
- Sign In link

Behavior:

- Real-time validation (email format, password strength)
- Show/hide password toggle
- Loading state on button
- Success message → Redirect to verification
- Error messages inline

4.1.3 Email Verification Page

URL: /verify-email

Layout:

Verify Your Email
We sent a verification link to: {{email}}
Click the link in your email to confirm your account.
[Resend Email]
Didn't receive email? Check spam

Features:

- Display email address
- Resend email link
- Auto-redirect after verification
- Expiration timer

4.1.4 Login Page

URL: /login

Layout:

DevSync	Sign In
Logo	
	Email:
	[_____]
	Password:
	[_____]
	[Remember Me]
	[Forgot Password?]
	[Sign In]
	or via [GitHub OAuth]
	Don't have account?
	[Sign Up]

Features:

- Email input
- Password input
- Remember me checkbox
- Forgot password link
- OAuth button
- Sign up link
- Error messages for failed login

4.1.5 Dashboard / Home Page

URL: /dashboard

Layout:

DevSync	[Notifications]	[Profile]	[Settings]
Sidebar	Main Content		
└ Home			
└ PRs	□ Welcome Back, User!		
└ Tasks			
└ Activity	□ Quick Stats		
└ Team			
└ Settings	3 PRs to Review 2 Tasks Assigned		
[Workspace]	5 New Notifications		
└ Project A	Last Activity: 2hrs		
└ Project B			
└ [+ New]			
	□ Recent Activity		
	└ PR #23 merged by Dev B (1h ago)		
	└ Task assigned to you (2h ago)		
	└ New PR created by Dev A (3h ago)		
	└ See All...		
	□ Pending Reviews		
	└ [Feature/login] by Dev A		
	└ [Fix/bug] by Dev C		

Sections:

- **Header:** Logo, notifications bell, user profile dropdown, settings
- **Left Sidebar:** Navigation menu, workspace selector
- **Main Content:**
 - Welcome message
 - Quick stats (PRs to review, tasks assigned, notifications)
 - Recent activity feed
 - Pending reviews section
 - Quick action buttons

Features:

- Notification badge (unread count)
- Quick stats clickable (go to that section)
- Activity feed real-time updates
- Responsive sidebar (collapsible on mobile)

4.1.6 Pull Requests Page

URL: /dashboard/prs

Layout:

DevSync [Notifications] [Profile] [Settings]	
Sidebar	Main Content
[Home]	
[PRs] ✓	Pull Requests
[Tasks]	
[Activity]	Filter: [All ▼] Search: [_____]
[Team]	Sort: [Newest] [□]
[Settings]	
	#25 Feature/Login by Dev A feature/login-fix Created: 2 hours ago Status: [OPEN] Comments: 3 Files: 5
	#24 Fix/Bug by Dev B fix/memory-leak Created: 5 hours ago Status: [MERGED] (1 hour ago) Comments: 7 Files: 3
	[Load More]

Components:

- **Filter Bar:** Status (Open/Merged/Closed), Author, Repo
- **Search:** By PR title or branch name
- **Sort:** Newest first/oldest first/most active
- **PR Card:** Shows number, title, author, branch, date, status, comment count, files changed
- **Status Badge:** Color-coded (Green=Merged, Blue=Open, Gray=Closed)
- **Pagination:** 20 PRs per page

Interactions:

- Click PR card → Open PR details page
 - Click author name → Filter by that author
 - Click status badge → Filter by that status
-

4.1.7 PR Details Page

URL: /dashboard/prs/:id

Layout:

DevSync	[Notifications] [Profile] [Settings]
Sidebar	PR Details
← Back	
	Feature/Login Fix
	#25 [OPEN]
	By Dev A feature/login-fix
	Created: 2 hours ago
	[View on GitHub]
	Description

	"Added user authentication..."
	Details

	Commits: 3
	Files Changed: 5
	Additions: +240 lines
	Deletions: -15 lines

	☐ Discussion (3 comments)
	[Dev B - 1 hour ago]
	"Looks good! Just one question..."
	[Dev A - 45 min ago]
	"Fixed that. Check the commit."
	[Dev C - 30 min ago]
	"Approved! Ready to merge."

	[Text Box] What's your feedback?
	[Send]

Sections:

- **PR Header:** PR number, title, status badge, author, branch, created date
- **View on GitHub Link:** Opens in new tab
- **Description:** Full PR description text
- **Details:** Commit count, files changed, lines added/removed
- **Discussion Thread:** All comments in chronological order
 - Each comment shows: author avatar, name, timestamp, message
 - Edit/delete buttons for own comments
- **Comment Input:** Text area to add new comment

Features:

- Real-time comment updates (if another user sends comment)
 - @mention support in comments
 - Markdown formatting in comments
 - Notification when PR is merged
 - Display merged timestamp if PR is merged
-

4.1.8 Tasks Page

URL: `/dashboard/tasks`

Layout:

DevSync	[Notifications]	[Profile]	[Settings]	
Sidebar	Main Content			
[Home]				
[PRs]	Tasks			
[Tasks] ✓				
[Activity]	[+ New Task]			
[Team]	Filter: [All ▼] [My Tasks] [Search]			
[Settings]				
	To Do		In Progress	
	Authentication		Dashboard	
	feature		Components	
	Assigned: You		Assigned:	
	Due: Mar 30		Dev B	
	☐ High Priority		Due: Mar 29	
			☐ Medium	
	Email		API	
	Reset Password		Optimization	
	Assigned:		Assigned:	
	Dev C		Dev A	
	Due: Apr 5		Due: Mar 31	
	☐ Medium		☐ Low	
	Done			
	Database Setup			
	Completed: 1d ago			
	✓			

View Types:

- **Kanban Board:** 3 columns (To Do, In Progress, Done)
- **List View:** Table format with task details
- **Toggle** between board and list view

Features:

- Drag-and-drop cards between columns
- Click card → Open task details
- Filter: Assigned to me, High priority, Due soon
- Search by task title
- Color-coded priority (Red=High, Yellow=Medium, Green=Low)
- Due date indicator
- Show assignee avatar

Card Information:

- Task title
- Assigned to (with avatar)
- Due date
- Priority badge
- Associated PR (if any)

4.1.9 Task Details Modal/Page

URL: `/dashboard/tasks/:id`

Layout:

Task Details	[X]
Task Title: Authentication Feature	
[_____]	
Description:	
[_____]	
[_____]	
Assigned To: [Dev A ▼]	
Priority: [High ▼]	
Status: [In Progress ▼]	
Due Date: [Mar 30, 2026 ▼]	
Related PR: [Feature/login #25]	
Comments:	
[Dev A - 2h ago]: "Starting work on this"	
[Text box] Add comment...	
[Save Changes] [Delete Task]	

Editable Fields:

- Title
- Description
- Assigned to (dropdown)
- Priority (High/Medium/Low)
- Status (To Do/In Progress/Done)
- Due date (date picker)
- Related PR (link to PR)

Read-Only Info:

- Created by
- Created date
- Last updated

- Update history

4.1.10 Activity Feed Page

URL: /dashboard/activity

Layout:

DevSync	[Notifications]	[Profile]	[Settings]
Sidebar	Main Content		
[Home]			
[PRs]	Team Activity		
[Tasks]			
[Activity] ✓	Filter: [All ▼] [Last 7 Days ▼]		
[Team]	Search: [_____]		
[Settings]			
	Today		
	2:30 PM		
	[] Dev A merged PR #25		
	"Feature/Login"		
	1:45 PM		
	[] Dev B commented on PR #24		
	"Looks good, approved!"		
	Yesterday		
	4:20 PM		
	[] Dev C created task		
	"Database optimization"		
	[Load More]		

Event Types:

- PR created
- PR merged
- PR closed
- Comment added
- Task created
- Task status changed
- Member added
- Member removed

Features:

- Grouped by date (Today, Yesterday, Last 7 Days, etc.)
 - Click event → Go to that entity (PR/task)
 - Filter by type and date range
 - Search by description
 - User avatar with event
 - Timestamp for each event
-

4.1.11 Team Management Page

URL: `/dashboard/team`

Layout:

DevSync	[Notifications]	[Profile]	[Settings]
Sidebar	Team Management		
[Home]			
[PRs]	Project: My Project		
[Tasks]			
[Activity]	☐ Team Members (3)		
[Team] ✓			
[Settings]	You (Owner) Email: you@example.com Joined: 5 days ago [Remove]		
	Dev A (Member) Email: deva@example.com Joined: 4 days ago [Remove]		
	Dev B (Member) Email: devb@example.com Joined: 3 days ago [Remove]		
	☐ Add Team Member		
	Email: [_____] Role: [Member ▼] [Send Invite]		
	☐ GitHub Repository		
	Repo: myteam/myproject Connected: Yes ✓ Webhook Installed: Yes ✓ [Disconnect]		

--	--	--

Sections:

1. Team Members List

- Member info: name, email, role, join date
- Remove button (owner only)

2. Add Team Member

- Email input
- Role selector
- Send invite button
- Shows "Invite Sent" message

3. GitHub Repository

- Show repo name
- Connection status
- Webhook status
- Disconnect button
- Option to reconnect

4.1.12 Notifications Page

URL: /dashboard/notifications

Layout:

DevSync	[Notifications] [Profile] [Settings]
Sidebar	Notifications
[Home]	
[PRs]	[Unread] [All] [Mark all as read]
[Tasks]	
[Activity]	Today
[Team]	
[Settings]	
	[Dev A pushed branch] "Feature/login-fix was pushed" 2 hours ago [View] [Mark as read]
	[Assigned task to you] "Authentication Feature" 1 hour ago [View] [Mark as read]
	[Dev B commented on PR] "Looks good!..." 30 minutes ago [View]
	Yesterday
	[PR merged] "Feature/login merged by Dev C" Yesterday at 5:30 PM [View]
	[Load More]

Features:

- Filter: Unread / All
- Mark individual notifications as read
- Mark all as read button
- Click notification → Go to related entity
- Grouped by date
- Notification types with icons:
 - ☐ PR notification (blue dot for unread)
 - ☐ Comment notification
 - ☐ Task notification

4.1.13 Settings Page

URL: /dashboard/settings

Layout:

DevSync	[Notifications] [Profile] [Settings]
Sidebar	Settings
[Home]	
[PRs]	[Account] [Notifications] [Privacy]
[Tasks]	
[Activity]	Account Settings
[Team]	
[Settings] ✓	
	Email:
	you@example.com
	Full Name:
	[You] [Save]
	Avatar:
	[Upload Photo] [Remove]
	Password:
	[Change Password]
	Connected GitHub:
	yourgithub [Disconnect]
	Delete Account:
	[Delete] (Permanent)
	Notification Preferences
	☒ PR Notifications (default)
	☒ Task Notifications
	☒ Comment Notifications
	☐ Daily Digest (instead of instant)
	Email Frequency:
	[Real-time ▼]
	(Options: Real-time, Daily,
	Weekly)
	[Save Preferences]

Tabs:

1. Account

- Email (read-only)
- Full name
- Avatar upload
- Change password
- Connected GitHub
- Delete account

2. Notifications

- Toggle notifications on/off
- Frequency (Real-time/Daily/Weekly)
- Per-type preferences (PR/Task/Comment)

3. Privacy

- Data deletion request
 - Account visibility
-

4.2 Mobile App Screens

4.2.1 Mobile Onboarding

Login Screen

DevSync Logo
Sign In
Email: [_____]
Password: [_____]
[Forgot Password?]
[SIGN IN]
or
[GitHub SIGN IN]
New here? [Sign Up]

4.2.2 Mobile Home/Dashboard

DevSync			
Good Morning, Dev!			
 Quick Stats			
• 3 PRs to Review			
• 2 Tasks Assigned			
• 5 Notifications			
 Recent Activity			
• PR #25 created by Dev A			
2 hours ago			
[View]			
• You assigned task to Dev B			
1 hour ago			
[View]			
• PR #24 merged			
Yesterday			
[View]			
[See All Activity]			
[Home]	[PRs]	[Tasks]	[More]

4.2.3 Mobile PR List

Pull Requests		☐	
[Open] [All] [Search]			
#25 Feature/Login			
by Dev A			
feature/login-fix			
Created: 2h ago		☐	
Comments: 3		Files: 5	
#24 Fix/Bug			
by Dev B			
fix/memory-leak			
Created: 5h ago		☒	
Comments: 7		Files: 3	
[Load More]			
[Home]		[PRs]	[Tasks]
		[More]	

4.2.4 Mobile PR Details

← #25 Feature/Login [...]		
Feature/Login		
[OPEN]		
by Dev A		
feature/login-fix		
2 hours ago		
[View on GitHub]		
Description		
"Added user auth..."		
Details		
• 3 commits		
• 5 files changed		
• +240 lines		
• -15 lines		
□ Comments (3)		
Dev B - 1h ago:		
"Looks good!"		
Dev A - 45m ago:		
"Fixed that."		
[Add Comment]		
[Comment...]		
[Home] [PRs] [Tasks] [More]		

4.2.5 Mobile Tasks

Tasks			[+ New]	[List]
☐ To Do (2)				
Authentication 1234				
Assigned: You				
Due: Mar 30			☐	
☐ High (v-scroll)				
Email Reset Pass...				
Assigned:				
Dev C				
Due: Apr 5				
☐ Medium				
▶ In Progress (1)				
▶ Done (4)				
[Home]	[PRs]	[Tasks]	[More]	

4.2.6 Mobile Activity Feed

Activity	[Today ▼]
Today	
□ Dev A merged PR #25	
2 hours ago	
[View PR]	
□ You assigned task	
1 hour ago	
[View Task]	
□ Dev B commented	
30 minutes ago	
[View]	
Yesterday	
□ PR #24 created	
Dev A on feature/auth	
[View PR]	
[Load More]	
[Home] [PRs] [Tasks] [More]	

4.2.7 Mobile Settings

Settings	[Account ▼]
Account Settings	
Email:	
you@example.com	(read-only)
Full Name:	
[You]	[Save]
Avatar:	
☐	[Change]
[Change Password]	
Connected GitHub:	
yourgithub	[Disconnect]
Notification Preferences	
☒	PR Notify
☒	Task Notify
☒	Comment Notify
☐	Daily Digest
Frequency:	
[Real-time ▼]	
(Options: Real-time, Daily)	
[Save]	
[Delete Account]	
[Sign Out]	
[Home] [PRs] [Tasks] [More]	

5. Workflows & Use Cases

5.1 Main Workflow: "Developer Pushes Code for Review"

Actors: Developer A, Developer B, Developer C (Team Members)

Preconditions:

- All team members have DevSync accounts
- GitHub repo is connected to DevSync
- Team members are added to project

Main Flow:

Step 1: Developer A Starts Work

- └ Developer A creates feature branch on GitHub
- └ Code for 2 hours on local machine
- └ Make commits with meaningful messages
- └ Push branch to GitHub

Step 2: GitHub Sends Webhook to DevSync

- └ GitHub detects push event
- └ Sends webhook payload to DevSync backend
- └ DevSync receives event
- └ Validates webhook signature
- └ Processes the event

Step 3: DevSync Creates PR Record

- └ Parse GitHub event data:
 - | └ Branch name: feature/login
 - | └ Author: Developer A
 - | └ Commits: 3
 - | └ Files changed: 5
 - | └ Timestamp
- └ Store in DevSync database
- └ Create notification record
- └ Set notification status: pending

Step 4: Email Notifications Sent

- └ DevSync triggers email service
- └ For each team member (Dev B, Dev C):
 - | └ Query user email from database
 - | └ Generate email template
 - | └ Include: PR title, branch, author, GitHub link, DevSync link
 - | └ Send via SendGrid
 - | └ Log email sent in database
- └ Notification marked as sent

Step 5: Team Members Receive Email

- └ Developer B gets email:
 - | -Subject: "[DevSync] New PR: Feature/login by Dev A"
 - | -Body: Shows PR details with links
- └ Developer C gets email:
 - | -Same content
- └ Email arrives within 5 minutes

Step 6: Developer B Opens DevSync

- └ Logs in to DevSync web app

- └ Dashboard shows new PR in feed
- └ Dashboard shows "1 PR to review"
- └ Clicks on PR
- └ Opens PR details page

Step 7: Developer B Reviews & Comments

- └ PR details page shows:
 - | └ PR title, branch, author
 - | └ Description
 - | └ Commit count, files changed
 - | └ "View on GitHub" button
 - | └ Discussion thread (empty initially)
- └ Developer B clicks "View on GitHub"
- └ Opens GitHub in new tab
- └ Reviews code on GitHub
- └ Comes back to DevSync
- └ Adds comment: "Looks good! Just one question about line 45."
- └ Comment sent and saved in DevSync database

Step 8: Developer A Notified of Comment

- └ DevSync trigger email: "Dev B commented on your PR #25"
- └ Developer A sees in app notifications
- └ Developer A clicks comment
- └ Sees comment from Developer B
- └ Replies in DevSync: "Fixed the issue. Check the latest commit."
- └ Comment saved and Developer B notified

Step 9: Developer C Reviews

- └ Developer C receives original email
- └ Opens DevSync
- └ Sees PR with 2 comments
- └ Reads discussion
- └ Adds comment: "All good! Ready to merge."
- └ All team members see this comment

Step 10: Developer A Merges on GitHub

- └ Developer A goes to GitHub PR page
- └ Clicks "Merge Pull Request"
- └ PR is merged to main branch
- └ GitHub sends merge webhook to DevSync

Step 11: DevSync Updates Status

- └ Receives merge event from GitHub
- └ Updates PR status: Open → Merged
- └ Stores merge timestamp

```
└─ Sends email: "PR #25 merged by Developer A"
└─ Updated PR details show: "Merged 5 minutes ago"
└─ Activity feed shows: "PR merged by Dev A"
└─ Task count on dashboard updates
```

Step 12: Complete Record Maintained

```
└─ DevSync shows full PR history:
|   └─ When created
|   └─ Who created it
|   └─ All comments and discussion
|   └─ When merged
|   └─ Who merged it
└─ All data searchable and filterable
└─ Team has complete record
└─ No information lost on WhatsApp
```

End: Workflow Complete ☐

5.2 Use Case: Task Assignment

Actors: Project Manager, Developer

Preconditions:

- Team is created
- Team members added
- User is admin or creator

Flow:

Step 1: Create Task

- └ Admin goes to Tasks page
- └ Click "+ New Task"
- └ Enter task title: "Set up Database"
- └ Enter description: "Setup PostgreSQL and create schema"
- └ Set assigned to: Developer A
- └ Set priority: High
- └ Set due date: March 30, 2026
- └ Click "Create Task"

Step 2: Task Saved

- └ Task stored in database
- └ Notification created
- └ Task appears in backlog
- └ Developer A assigned

Step 3: Notification Sent

- └ Email sent to Developer A
- └ Subject: "New task: Set up Database"
- └ Body: Shows task details and due date
- └ Email delivered

Step 4: Developer A Sees Task

- └ Opens DevSync or receives email
- └ Sees task in "My Tasks" section
- └ Reads task details
- └ Acknowledges and starts work

Step 5: Developer A Updates Status

- └ Goes to Tasks page
- └ Clicks on "Set up Database"
- └ Changes status: To Do → In Progress
- └ Saves
- └ Task moves to "In Progress" column

Step 6: Notification Sent

- └ Email to admin: "Dev A started task: Set up Database"
- └ Activity feed updated
- └ Team sees Developer A is working on it

Step 7: Developer A Completes

- └ Task completed
- └ Changes status: In Progress → Done
- └ Adds comment: "Database setup complete. Repo updated."

Step 8: Final Notification

- └ Email to admin: "Task completed by Dev A"
- └ Task moves to "Done" column
- └ Activity shows task completion
- └ Record maintained

End: Task Complete □

5.3 Key Workflow Points

Critical Paths:

1. GitHub → DevSync → Email → User

- GitHub sends webhook
- DevSync processes instantly
- Email sent within 5 minutes
- User sees notification

2. User → DevSync → Notification

- User takes action (comment, status update)
- Saved immediately
- Relevant users notified via email
- History maintained

3. Team Coordination

- All communication in one place
- No missed messages on WhatsApp
- Complete history for reference
- Professional record

6. Data Models

6.1 Database Tables

users

```

CREATE TABLE users (
  id INT PRIMARY KEY AUTO_INCREMENT,
  email VARCHAR(255) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  full_name VARCHAR(255),
  avatar_url VARCHAR(255),
  github_username VARCHAR(255),
  github_id INT UNIQUE,
  email_verified BOOLEAN DEFAULT FALSE,
  email_verified_at TIMESTAMP,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  deleted_at TIMESTAMP NULL
);

```

teams

```

CREATE TABLE teams (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(255) NOT NULL,
  description TEXT,
  avatar_url VARCHAR(255),
  created_by INT NOT NULL REFERENCES users(id),
  github_repo_owner VARCHAR(255),
  github_repo_name VARCHAR(255),
  webhook_secret VARCHAR(255),
  webhook_installed BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  deleted_at TIMESTAMP NULL
);

```

team_members

```

CREATE TABLE team_members (
  id INT PRIMARY KEY AUTO_INCREMENT,
  team_id INT NOT NULL REFERENCES teams(id),
  user_id INT NOT NULL REFERENCES users(id),
  role VARCHAR(50) DEFAULT 'member', -- 'admin', 'member'
  joined_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE(team_id, user_id)
);

```

pull_requests

```
CREATE TABLE pull_requests (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  team_id INT NOT NULL REFERENCES teams(id),  
  github_pr_id INT NOT NULL,  
  github_repo_name VARCHAR(255),  
  pr_number INT,  
  title VARCHAR(255) NOT NULL,  
  description TEXT,  
  branch_name VARCHAR(255) NOT NULL,  
  author_github_username VARCHAR(255) NOT NULL,  
  author_name VARCHAR(255),  
  author_avatar_url VARCHAR(255),  
  commit_count INT DEFAULT 0,  
  files_changed INT DEFAULT 0,  
  status VARCHAR(50), -- 'open', 'merged', 'closed'  
  github_pr_url VARCHAR(500),  
  created_at TIMESTAMP,  
  merged_at TIMESTAMP NULL,  
  merged_by VARCHAR(255) NULL,  
  closed_at TIMESTAMP NULL,  
  synced_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  UNIQUE(team_id, github_pr_id)  
);
```

pr_comments

```
CREATE TABLE pr_comments (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  pr_id INT NOT NULL REFERENCES pull_requests(id),  
  user_id INT NOT NULL REFERENCES users(id),  
  message TEXT NOT NULL,  
  edited_at TIMESTAMP NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  deleted_at TIMESTAMP NULL  
);
```

tasks

```

CREATE TABLE tasks (
  id INT PRIMARY KEY AUTO_INCREMENT,
  team_id INT NOT NULL REFERENCES teams(id),
  title VARCHAR(255) NOT NULL,
  description TEXT,
  assigned_to INT NOT NULL REFERENCES users(id),
  created_by INT NOT NULL REFERENCES users(id),
  status VARCHAR(50) DEFAULT 'todo', -- 'todo', 'in_progress', 'done'
  priority VARCHAR(50) DEFAULT 'medium', -- 'low', 'medium', 'high'
  due_date DATE NULL,
  related_pr_id INT NULL REFERENCES pull_requests(id),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  completed_at TIMESTAMP NULL,
  deleted_at TIMESTAMP NULL
);

```

notifications

```

CREATE TABLE notifications (
  id INT PRIMARY KEY AUTO_INCREMENT,
  user_id INT NOT NULL REFERENCES users(id),
  team_id INT NOT NULL REFERENCES teams(id),
  type VARCHAR(100), -- 'pr_created', 'pr_merged', 'comment_added', 'task_assigned', etc.
  related_pr_id INT NULL REFERENCES pull_requests(id),
  related_task_id INT NULL REFERENCES tasks(id),
  actor_user_id INT NULL REFERENCES users(id),
  title VARCHAR(255),
  message TEXT,
  email_sent BOOLEAN DEFAULT FALSE,
  email_sent_at TIMESTAMP NULL,
  read_at TIMESTAMP NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

notification_preferences

```
CREATE TABLE notification_preferences (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  user_id INT NOT NULL REFERENCES users(id),  
  pr_notifications BOOLEAN DEFAULT TRUE,  
  task_notifications BOOLEAN DEFAULT TRUE,  
  comment_notifications BOOLEAN DEFAULT TRUE,  
  use_daily_digest BOOLEAN DEFAULT FALSE,  
  digest_time TIME DEFAULT '09:00:00',  
  frequency VARCHAR(50) DEFAULT 'realtime', -- 'realtime', 'daily', 'weekly'  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP  
);
```

activity_logs

```
CREATE TABLE activity_logs (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  team_id INT NOT NULL REFERENCES teams(id),  
  actor_user_id INT NOT NULL REFERENCES users(id),  
  action_type VARCHAR(100), -- 'pr_created', 'pr_merged', 'comment', 'task_created', etc.  
  entity_type VARCHAR(100), -- 'pr', 'task', 'member'  
  entity_id INT,  
  description TEXT,  
  metadata JSON, -- Store additional data as JSON  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  INDEX(team_id, created_at)  
);
```

7. Non-Functional Requirements

7.1 Performance

- **Response Time:** < 2 seconds for web pages
- **Email Delivery:** < 5 minutes from event
- **Webhook Processing:** < 1 second
- **Database Queries:** < 500ms
- **Concurrent Users:** Support 100+ simultaneous users per team
- **Data Handling:** Support 10,000+ PRs per team

7.2 Reliability & Availability

- **Uptime:** 99.9% SLA (max 45 minutes downtime per month)
- **Backup:** Automatic daily backups, 30-day retention

- **Disaster Recovery:** RTO < 1 hour, RPO < 15 minutes
- **Webhook Resilience:** Retry logic with exponential backoff
- **Error Handling:** Graceful degradation if email service fails

7.3 Security

- **Data Transit:** All data encrypted with HTTPS/TLS 1.2+
- **At Rest:** Database encryption with AES-256
- **Authentication:** Secure password hashing (bcrypt)
- **GitHub Webhook:** Signature validation on every webhook
- **Password Policy:** Minimum 8 characters, uppercase, numbers, symbols
- **Session:** 30-minute timeout, secure cookies
- **Rate Limiting:** 100 API calls per minute per user
- **Injection Prevention:** Parameterized queries, input sanitization
- **XSS Protection:** Content Security Policy headers
- **GDPR Compliance:** Data deletion on user request
- **Audit Logs:** All data changes logged

7.4 Scalability

- **Database:** Horizontal replication, read replicas
- **Backend:** Cloud-native, auto-scaling
- **Static Assets:** CDN distribution
- **Caching:** Redis for frequently accessed data
- **Message Queue:** Async processing for emails
- **Load Balancing:** Distribute traffic across instances

7.5 Maintainability

- **Code Quality:** Clean code, DRY principles
- **Testing:** 80% unit test coverage, integration tests
- **Documentation:** API docs, deployment guides, runbooks
- **Logging:** Structured logging to centralized service
- **Monitoring:** Real-time alerts for errors, performance issues
- **CI/CD:** Automated tests on every commit
- **Version Control:** Clear commit messages, feature branches

7.6 Usability

- **UI Responsiveness:** Works on desktop, tablet, mobile
 - **Accessibility:** WCAG 2.1 Level AA compliance
 - **Loading States:** Clear visual feedback during loading
 - **Error Messages:** Clear, actionable error messages
 - **Onboarding:** Guided setup for new users
-

8. Assumptions & Constraints

8.1 Assumptions

1. Users have GitHub accounts and repositories
2. Users have valid email addresses
3. Users have internet connection
4. GitHub API is available and responsive
5. Email service (SendGrid) is operational
6. Team members are trustworthy (no permission-based security)
7. PR data is public or accessible to authorized users
8. Users prefer email notifications (can be changed in v2)

8.2 Constraints

1. **GitHub Integration:** Only supports public and private repos accessible to authenticated user
2. **Email:** Limited to email-based notifications (SMS in v2)
3. **PRs:** Only tracks PRs from connected GitHub repos
4. **User Limit:** Maximum 50 members per team (MVP limit)
5. **Task Limit:** Maximum 50 active tasks per team
6. **Storage:** Maximum 1GB per team (for logs, media)
7. **Uptime:** No guarantee on GitHub/email service availability
8. **Browser Support:** Chrome, Firefox, Safari, Edge (latest versions)
9. **Mobile:** iOS 12+ and Android 8+ for mobile app
10. **Deployment:** Initially cloud-only (AWS/Heroku/Azure)

8.3 Future Enhancements (v2+)

- ☐ Slack/Discord integration
- ☐ SMS notifications
- ☐ Advanced analytics dashboard
- ☐ Code review automation
- ☐ Integration with Jira/Linear
- ☐ Dark mode UI
- ☐ Multi-language support
- ☐ AI-powered code insights
- ☐ Self-hosted option
- ☐ Video call integration

Conclusion

This SRS document provides complete specification for building DevSync. The system is designed to:

1. **Solve a Real Problem:** Replace WhatsApp for dev coordination
2. **Integrate Seamlessly:** Work with existing GitHub workflow
3. **Keep Data Organized:** Central hub for all team communication
4. **Scale Efficiently:** Support growing teams
5. **Remain Simple:** Core features only, no complexity

Next Steps:

- Approve this SRS
- Begin database design
- Set up development environment
- Create UI mockups
- Start backend development

Document Version: 1.0

Last Updated: March 27, 2026

Status: Ready for Development